DOMAIN 5 // POST-EXPLOITATION & LATERAL MOVEMENT

COMPTIA PENTEST+ PT0-003 · PERSISTENCE · LATERAL MOVEMENT · EXFILTRATION · CLEANUP

5.0 = 14% OF EXAM
~13 QUESTIONS

THE SHAPE OF DOMAIN 5 READ FIRST

This is **after the breach**. You already have a foothold — now the exam tests what a professional does next, in order: **stay in** (5.1 persistence) → **move sideways** (5.2 lateral movement) → **get the data out** (5.3 exfiltration) → **leave it clean** (5.4 cleanup). The recurring theme is *quiet and reversible* — blend in, and undo everything you did.

> THE TWO CANONICAL DOMAIN 5 TOOLS
>
> **Mimikatz** (dump creds to fuel the next hop) and **Impacket** (the Python AD/SMB glue for remote exec, relaying, ticket abuse). **CrackMapExec** is the third — validate creds and move across many hosts at once. If one of these fits, it's usually the answer.

5.1 ESTABLISH & MAINTAIN PERSISTENCE GIVEN A SCENARIO, PERFORM

Goal: a way *back in* that survives reboot and credential change, plus a channel to *task* the host.

TECHNIQUE	WHAT IT DOES / EXAMPLE	KEYWORD
Scheduled tasks / cron	Time- or event-triggered re-execution — e.g. a fake "Windows Update" task at boot	"survives reboot", "runs on a schedule/at startup"
Service creation	A new auto-start Windows service running your binary	"auto-start service", "runs as SYSTEM"
Registry keys	Run / RunOnce autostart entries	"Run key", "HKCU/HKLM ...\\Run"
Reverse shell	Target dials *out* to you — beats inbound firewalls & NAT	"egress only", "firewalled inbound" → **reverse**
Bind shell	Target opens a listener and waits for you	"target listens" — fails against a perimeter firewall
New accounts / valid creds	A rogue local/domain account — the quietest, most durable persistence	"add a user", "obtain valid credentials"
C2 framework	Centralized tasking of the foothold	"command and control" → **Covenant, Metasploit**
Backdoor (web shell / trojan)	Web shell on a server; trojanized binary on a host	"web shell", "trojanized"
Rootkit	Kernel/boot-level hiding — deepest, hardest to remove	"kernel-level", "hardest to detect/remove"
Browser extensions	Persistence + data theft through the browser	"malicious extension"
Tampering security controls	Disable AV/EDR/logging to protect the foothold	"disable Defender/logging"

> EXAMPLE — REVERSE VS BIND SHELL The target sits behind a firewall that blocks all inbound connections but allows outbound 443. A BIND SHELL (target listens on a port) is unreachable — the firewall drops your inbound connection. A REVERSE SHELL has the target connect *out* to your listener on 443, which the firewall permits as normal web traffic. Whenever the scenario stresses inbound filtering, egress-only, or NAT, the answer is reverse.

> EXAM TELLS
>
> "Maintain access after reboot" → scheduled task / service / Run key (not a plain in-memory shell, which dies on reboot). "Quietest long-term access" → a valid account. "Deepest hiding" → rootkit. "Get past the inbound firewall" → reverse shell. Persistence you create is something you must *remove* in 5.4 — a named obligation.

5.2 LATERAL MOVEMENT — GIVEN A SCENARIO, PERFORM

MOVE, PIVOT & ENUMERATE

CONCEPT	WHAT IT DOES / EXAMPLE	KEYWORD & TOOL
Pivoting	Route your traffic *through* the compromised host to reach a segment you can't touch directly	"can't route to it directly" → **sshuttle, Proxychains**
Relay creation	Set up a relay/redirector through the foothold	"relay through the host"
Remote execution	Run commands on the next host with valid creds	"execute on a remote host" → **PsExec, Impacket, WinRM, WMI**
Credential dumping	Harvest more creds on each new host to keep moving	"dump creds to pivot" → **Mimikatz**
Enumeration	Service/traffic discovery, additional cred capture, string searches	"find the next hop from the foothold"

SERVICE-DISCOVERY TARGETS & NATIVE REMOTING

SERVICE / MECHANISM	WHY IT MATTERS FOR LATERAL MOVEMENT	KEYWORD
SMB / fileshares	Shares hold creds & enable remote exec (PsExec)	"port 445", "admin share"
RDP / VNC	Interactive remote desktop	"3389", "remote desktop"
SSH	Remote shell / tunnel on Linux	"port 22", "key reuse"
WMI / WinRM	Native Windows remoting — blends into admin traffic	"blend in", "native admin", "5985"
LDAP / RPC / DCOM	Directory & RPC-based movement	"query the DC", "DCOM object"
LPD / JetDirect	Printer protocols — overlooked pivot points	"printer", "port 515/9100"
FTP / Telnet / cleartext	Legacy protocols leaking creds	"cleartext credentials on the wire"

> EXAMPLE — PIVOTING THROUGH A FOOTHOLD You own a web server in a DMZ that can reach the internal `10.0.5.0/24`, but your attack box cannot. You run `SSHUTTLE` over your SSH session to the web server — now your box transparently routes internal traffic through it, and you scan and attack `10.0.5.x` as if you were on that segment. If you only need one tool routed, `PROXYCHAINS` (e.g. `proxychains nmap -sT 10.0.5.10`) does it per-tool instead.

> EXAMPLE — HARVEST-AND-HOP On host A you dump creds with Mimikatz and recover a domain admin's NTLM hash. You `PASS-THE-HASH` with CrackMapExec across the subnet, find the hash is valid on host B, and use `IMPACKET'S PSEXEC.PY` to get SYSTEM on B. Each hop harvests more material for the next — the loop that defines lateral movement.

> TWO DISCRIMINATORS THEY TEST
>
> **sshuttle vs Proxychains** — sshuttle = transparent VPN-like tunnel catching all traffic; Proxychains = routes a specific tool through a chain. **PsExec vs Impacket's psexec.py** — same job (remote exec with creds); PsExec is the native SysInternals Windows tool, psexec.py the Python/Linux-attacker version. Answer on the OS/tooling in the stem.

> NOTE ON SCANS THROUGH A PROXY
>
> Half-open SYN scans don't work through Proxychains — use a full TCP-connect scan (`nmap -sT`). A scenario pairing "pivot" with a scan is often testing this.

5.2 LIVING OFF THE LAND BINARIES — NATIVE, SIGNED, ALLOW-LISTED

BINARY	PURPOSE	KEYWORD
rundll32	Executes exported DLL functions	**most-abused** — direct execution, AppLocker bypass
msbuild	Compiles & runs inline C#	allow-list bypass via a trusted build tool
mmc.exe	Loads admin snap-ins	proxied execution via .msc/COM
Net commands	User/group/share management	`net user`, `net group` — enum, no extra tools
Netstat / route	Connections & routing table	recon for pivoting
strings / findstr	Search files for patterns	hunt creds in config/log files
cmd.exe / explorer.exe / ftp.exe	Native shell / launch / transfer	baseline exec & exfil with trusted binaries

> THE LOLBIN TELL
>
> "Bypass the allow-list / run blocked code with a trusted binary" → **rundll32** or **msbuild** specifically. The rest (netstat, net, route, strings) are recon-oriented. If the verb is *execute* rather than *gather*, lean rundll32/msbuild. Why they work: the binary is Microsoft-signed and already allow-listed, so the control never fires.

5.2 LATERAL-MOVEMENT TOOLS & FRAMEWORKS — USE THE APPROPRIATE

TOOL	PURPOSE	KEYWORD
Mimikatz	Dump creds/hashes/tickets from LSASS	"extract from memory" to fuel the next hop
Impacket	Python AD/SMB toolkit (psexec.py, wmiexec.py, ntlmrelayx)	the glue tool — remote exec, relay, ticket abuse
CrackMapExec	Validate creds / spray / exec across many hosts	"test creds across the subnet", "move broadly"
PsExec	Remote process execution (SysInternals)	"run remotely with valid creds" (native Windows)
Covenant	.NET C2 framework	persistent C2 tasking (Metasploit is the other)
Netcat	General net utility	reverse/bind shells, quick transfer
sshuttle	Transparent VPN-like tunnel over SSH	"pivot all traffic through the host"
Proxychains	Route a specific tool through a proxy chain	"pivot this one tool", `-sT` required
PowerShell ISE / Batch	On-host scripting & automation	"run/edit a script on the host"

> **CORRECTION & CANON**
> The C2 answers on PT0-003 are **Covenant** and **Metasploit** — "Empire" is not on the objectives. For any "move to another host with discovered credentials" scenario, the workhorses are **CrackMapExec** and **Impacket**.

5.3 STAGING & EXFILTRATION — SUMMARIZE

CONCEPT	WHAT IT DOES / EXAMPLE	KEYWORD
File encryption & compression	Stage data before it leaves — shrink & hide it	"compress and encrypt before exfil"
DNS tunneling	Encode data in DNS queries — rarely blocked outbound	"exfil via DNS", "no direct internet but DNS resolves"
ICMP tunneling	Encode data in ping payloads	"exfil over ICMP/ping"
HTTPS / covert channel	Blend exfil into normal encrypted web traffic	"looks like normal web traffic"
Steganography	Hide data inside an image/file	"hidden in an image"
Alternate data streams	Hide data in NTFS with no visible size change	"NTFS", "no size change", `file.txt:hidden`
Cloud / text-storage / email	Exfil destinations — cross-account buckets, paste sites, mail	"upload to a paste site / cloud bucket"

> **EXAMPLE — DNS TUNNELING WHEN NOTHING ELSE GETS OUT** A segmented host has no direct internet route, but it can resolve DNS through the internal resolver. You encode stolen data into subdomain labels — `<base32-chunk>.exfil.attacker.com` — and your authoritative name server reassembles the queries. Firewalls almost never block outbound DNS, which is exactly why it's the classic covert channel.

> **EXAM TELLS**
> "Exfiltrate without tripping the firewall" → **DNS or ICMP tunneling**. "Hide data in NTFS with no visible size change" → **alternate data streams**. "Hide inside an image" → **steganography**. "Make it look like normal traffic" → **HTTPS covert channel**. Stage first: compress + encrypt before moving anything.

5.4 CLEANUP & RESTORATION — EXPLAIN

ACTIVITY	WHY IT MATTERS / WHAT TO EXPECT
Remove persistence mechanisms	delete every task, service, account & key you created — the #1 cleanup obligation
Revert configuration changes	put altered settings (disabled AV, firewall rules) back exactly
Remove tester-created credentials	no rogue accounts left behind that anyone could reuse
Remove tools	delete uploaded binaries, scripts, payloads from every host
Spin down infrastructure	tear down C2, redirectors, phishing sites
Preserve artifacts	**keep** the evidence your report needs — screenshots, logs, proof
Securely destroy collected data	destroy client data you gathered per the agreed retention terms

> **EXAMPLE — THE TENSION THE EXAM TESTS** You created a scheduled task, added a local admin, disabled Defender, and exfiltrated a database of PII to prove impact. Cleanup means: REMOVE the task and the account, RE-ENABLE Defender, DELETE your tools — but PRESERVE the screenshots proving you did it, and SECURELY DESTROY the PII you copied once the report is written. "Leave it as you found it" never means "destroy your evidence".

> **THE BRIDGE THAT MAKES THIS TESTABLE**
> Cleanup is an **ethical and contractual obligation**, not optional — it ties directly to data handling in the RoE (1.1) and legal/ethical reporting considerations (1.4). The trap answer either forgets to remove persistence (leaving the client exposed) or destroys the artifacts needed for the report. Correct = remove your changes, preserve your evidence, destroy the client's data you collected.

Reverse shell beats the firewall. // Mimikatz + Impacket = harvest and hop. // DNS tunnels the data out. // Remove your changes, preserve the evidence.